

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ  
РОССИЙСКОЙ ФЕДЕРАЦИИ

ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ  
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ

**«БЕЛГОРОДСКИЙ ГОСУДАРСТВЕННЫЙ ТЕХНОЛОГИЧЕСКИЙ  
УНИВЕРСИТЕТ им. В. Г. Шухова»  
(БГТУ им. В. Г. Шухова)**



Кафедра программного обеспечения вычислительной  
техники и автоматизированных систем

**Лабораторная работа №1**

по дисциплине: «Параллельное программирование»  
на тему: «Сравнение парадигм конкурентности и параллелизма при  
разработке многопоточных программ в ОС Linux.»

Выполнил: ст. группы ПВ-223  
Дмитриев Андрей Александрович

Проверили:  
доц. Островский Алексей Мичеславович,

Белгород, 2025

**Цель работы:** Исследовать чувствительность вычислительной схемы из индивидуального задания к:

- а) ситуациям конкурентности, когда несколько потоков разделяют одно процессорное ядро.
- б) ситуациям параллелизма, когда каждый поток выполняется на отдельном ядре процессора (нет конкуренции за вычислительные ресурсы).

Цель работы обуславливает постановку и решение следующих **задач**:

- 1) Изучить основные принципы многопоточного программирования в ОС Linux, включая различия между конкурентностью и параллелизмом.
- 2) Получить навыки работы с POSIX Threads (pthread) и инструментами управления потоками, такими как sched\_setaffinity и taskset.
- 3) Ознакомиться с механизмами планирования потоков, управления процессорным временем и анализа производительности многопоточных программ.
- 4) Выполнить индивидуальное задание, связанное с использованием POSIX Threads для реализации вычислительной задачи с контролируемым распределением потоков по процессорным ядрам. Следует декомпозировать вычислительную задачу, вычленив сущности для потоковой обработки.
- 5) Построить графики зависимости вычислительной эффективности программы от числа потоков для ситуаций (а) и (б), проанализировать накладные расходы, связанные с переключением контекста, оценить влияние гиперпоточности.

**Условие индивидуального задания:**

Произвести расчёт следующего выражения:

$$S = \sum_{i=1}^N \frac{\sin(i) + \cos(2i) + i^3}{\sqrt{i+1} + \ln(i+1)}$$

## Ход выполнения работы

### Текст программы на языке C.

test\_concurrency.c

```
#define _GNU_SOURCE

#include <pthread.h>
#include <stdio.h>
#include <sched.h>
#include <unistd.h>
#include <stdlib.h>
#include <stdint.h>
#include <math.h>

#define NUM_THREADS 4

uint64_t func(size_t num_iter)
{
    volatile uint64_t sum = 0;
    for (size_t i = 0; i < num_iter; i++)
    {
        double dividend = sin(i) + cos(2 * i) + pow(i, 3);
        double divisor = sqrt(i + 1) + log(i + 1);
        sum += (uint64_t)(dividend / divisor);
    }

    return sum;
}

void *compute(void *arg)
{
    volatile uint64_t num_iters = (uint64_t)arg;
    func(num_iters);
}

int main(int argc, char const *argv[])
{
    uint64_t num_iters = atoll(argv[1]);

    pthread_t threads[NUM_THREADS];
    for (size_t iter = 0; iter < NUM_THREADS; iter++)
        pthread_create(&threads[iter], NULL, compute, (void *)num_iters);

    for (size_t iter = 0; iter < NUM_THREADS; iter++)
        pthread_join(threads[iter], NULL);

    return 0;
}
```

## test\_parallel.c

```
#define _GNU_SOURCE

#include <pthread.h>
#include <stdio.h>
#include <sched.h>
#include <unistd.h>
#include <stdlib.h>
#include <stdint.h>
#include <math.h>

#define NUM_THREADS 4

struct args
{
    int core_id;
    uint64_t num_iters;
};

uint64_t func(size_t num_iter)
{
    volatile uint64_t sum = 0;
    for (size_t i = 0; i < num_iter; i++)
    {
        double dividend = sin(i) + cos(2 * i) + pow(i, 3);
        double divisor = sqrt(i + 1) + log(i + 1);
        sum += (uint64_t)(dividend / divisor);
    }

    return sum;
}

// Принудительное закрепление потока за конкретным ядром
void pin_thread_to_core(int core_id)
{
    cpu_set_t cpuset;
    CPU_ZERO(&cpuset);
    CPU_SET(core_id, &cpuset);
    pthread_t current_thread = pthread_self();
    pthread_setaffinity_np(current_thread, sizeof(cpu_set_t), &cpuset);
}

void *compute_pinned(void *args)
{
    struct args *args_by_struct = (struct args *)args;

    int core_id = args_by_struct->core_id;
    pin_thread_to_core(core_id);

    uint64_t num_iters = args_by_struct->num_iters;
```

```

    func(num_iters);
}

int main(int argc, char const *argv[])
{
    uint64_t num_iters = atoll(argv[1]);
    printf("%lu\n", num_iters);

    pthread_t threads[NUM_THREADS];
    for (size_t iter = 0; iter < NUM_THREADS; iter++)
    {
        struct args args = {iter, num_iters};
        pthread_create(&threads[iter], NULL, compute_pinned, (void *)&args);
    }

    for (size_t iter = 0; iter < NUM_THREADS; iter++)
        pthread_join(threads[iter], NULL);

    return 0;
}

```

### run\_test\_concurrency.sh

```

gcc ./test_concurrency.c -pthread -lm -o t.o

if [ ! -f ./t.o ]
then
    echo compile error
    exit 1
fi

# Имя CSV-файла
output_csv="timetest_concurrency.csv"

# Заголовок CSV-файла
echo "sep=;" > "$output_csv"
echo "input_arg;real_time;user_time;sys_time" >> "$output_csv"

# Функция для преобразования времени в секунды
convert_to_seconds() {
    local time_str=$1
    # Убираем 'm' и 's', затем разбиваем на минуты и секунды
    local minutes=$(echo "$time_str" | sed 's/m.*//g')
    local seconds=$(echo "$time_str" | sed 's/.*m//g; s/s//g')
    # Вычисляем общее время в секундах
    echo "$minutes * 60 + $seconds" | bc
}

# Функция для выполнения эксперимента
run_experiment() {
    local input_arg=$1

```

```

echo "Эксперимент 'Ситуация конкурентности' с кол-вом итераций: $input_arg"

# Замер времени выполнения команды sleep с переданным аргументом
output=$( { time taskset -c 0 ./t.o $input_arg; } 2>&1 )

# Извлечение real, user и sys времени из вывода time
real_time=$(echo "$output" | grep real | awk '{print $2}')
user_time=$(echo "$output" | grep user | awk '{print $2}')
sys_time=$(echo "$output" | grep sys | awk '{print $2}')

# Преобразование времени в секунды
real_seconds=$(convert_to_seconds "$real_time")
user_seconds=$(convert_to_seconds "$user_time")
sys_seconds=$(convert_to_seconds "$sys_time")

# Вывод результатов в консоль
echo "real_time: $real_seconds секунд"
echo "user_time: $user_seconds секунд"
echo "sys_time: $sys_seconds секунд"
echo "-----"

# Запись результатов в CSV-файл
echo "$input_arg;$real_seconds;$user_seconds;$sys_seconds" >> "$output_csv"
}

# Выполнение трех экспериментов
for (( i=$1; i<=$2; i+=3 ))
do
    run_experiment $i
done

rm t.o

echo "Результаты сохранены в файл: $output_csv"

```

run\_test\_parallel.sh

```
gcc ./test_parallel.c -pthread -lm -o t.o
```

```

if [ ! -f ./t.o ]
then
    echo compile error
    exit 1
fi

```

# Имя CSV-файла

```
output_csv="timetest_parallel.csv"
```

# Заголовок CSV-файла

```
echo "sep=;" > "$output_csv"
```

```
echo "input_arg;real_time;user_time;sys_time" >> "$output_csv"
```

```

# Функция для преобразования времени в секунды
convert_to_seconds() {
    local time_str=$1
    # Убираем 'm' и 's', затем разбиваем на минуты и секунды
    local minutes=$(echo "$time_str" | sed 's/m.*//g')
    local seconds=$(echo "$time_str" | sed 's/.*m//g; s/s//g')
    # Вычисляем общее время в секундах
    echo "$minutes * 60 + $seconds" | bc
}

# Функция для выполнения эксперимента
run_experiment() {
    local input_arg=$1
    echo "Эксперимент 'Ситуация параллелизма' с кол-вом итераций: $input_arg"

    # Замер времени выполнения команды sleep с переданным аргументом
    output=$( { time ./t.o $input_arg; } 2>&1 )

    # Извлечение real, user и sys времени из вывода time
    real_time=$(echo "$output" | grep real | awk '{print $2}')
    user_time=$(echo "$output" | grep user | awk '{print $2}')
    sys_time=$(echo "$output" | grep sys | awk '{print $2}')

    # Преобразование времени в секунды
    real_seconds=$(convert_to_seconds "$real_time")
    user_seconds=$(convert_to_seconds "$user_time")
    sys_seconds=$(convert_to_seconds "$sys_time")

    # Вывод результатов в консоль
    echo "real_time: $real_seconds секунд"
    echo "user_time: $user_seconds секунд"
    echo "sys_time: $sys_seconds секунд"
    echo "-----"

    # Запись результатов в CSV-файл
    echo "$input_arg;$real_seconds;$user_seconds;$sys_seconds" >> "$output_csv"
}

# Выполнение трех экспериментов
for (( i=$1; i<=$2; i+=3 ))
do
    run_experiment $i
done

rm t.o

echo "Результаты сохранены в файл: $output_csv"

```

## output\_graph.py

```
import pandas as pd
import matplotlib.pyplot as plt

def output_graph(path: str, name: str):
    # Загрузка данных из CSV-файла
    data = pd.read_csv(path, sep=";", skiprows=2,
                       names=["input_arg", "real_time", "user_time",
                              "sys_time"])

    # Построение графиков
    plt.figure(figsize=(10, 6))

    # График real_time
    plt.plot(data["input_arg"], data["real_time"], label="real_time",
             marker="o")

    # График user_time
    plt.plot(data["input_arg"], data["user_time"], label="user_time",
             marker="s")

    # График sys_time
    plt.plot(data["input_arg"], data["sys_time"], label="sys_time",
             marker="^")

    # Настройка графика
    plt.title(f"{name}. Зависимость времени выполнения от количества итераций.")
    plt.xlabel("Количество итераций (input_arg)")
    plt.ylabel("Время (секунды)")
    plt.legend() # Добавление легенды
    plt.grid(True) # Включение сетки

    # Сохранение графика в файл
    plt.savefig(f"{name}.png")

    # Показать график
    plt.show()

output_graph("./timetest_parallel.csv", "Параллелизм")
output_graph("./timetest_concurrency.csv", "Конкурентность")
```

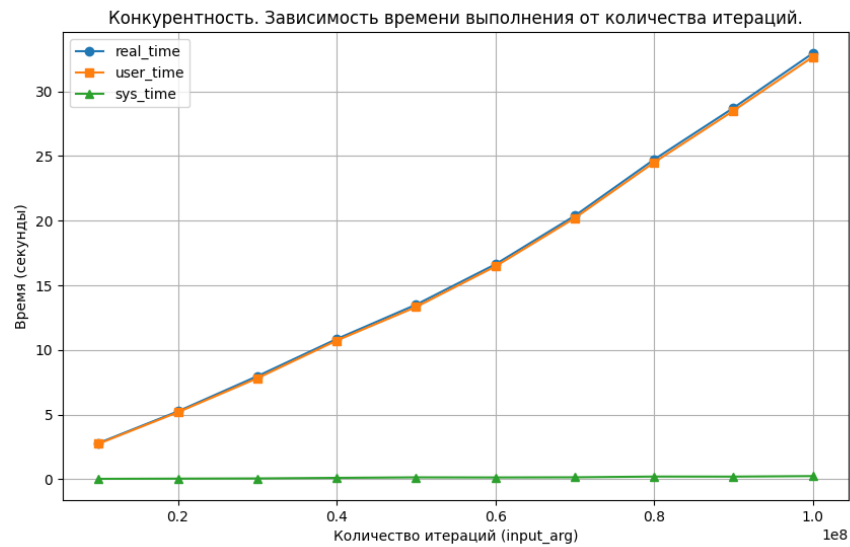
## Протоколы, логи, скриншоты, графики.

### timetest\_concurrency

```
sep=;
input_arg;real_time;user_time;sys_time
10000000;2.800;2.748;.038
20000000;5.254;5.187;.056
30000000;7.971;7.813;.072
40000000;10.847;10.730;.115
50000000;13.503;13.324;.154
60000000;16.625;16.465;.146
70000000;20.378;20.202;.158
80000000;24.732;24.508;.214
90000000;28.726;28.505;.211
100000000;32.949;32.669;.252
```



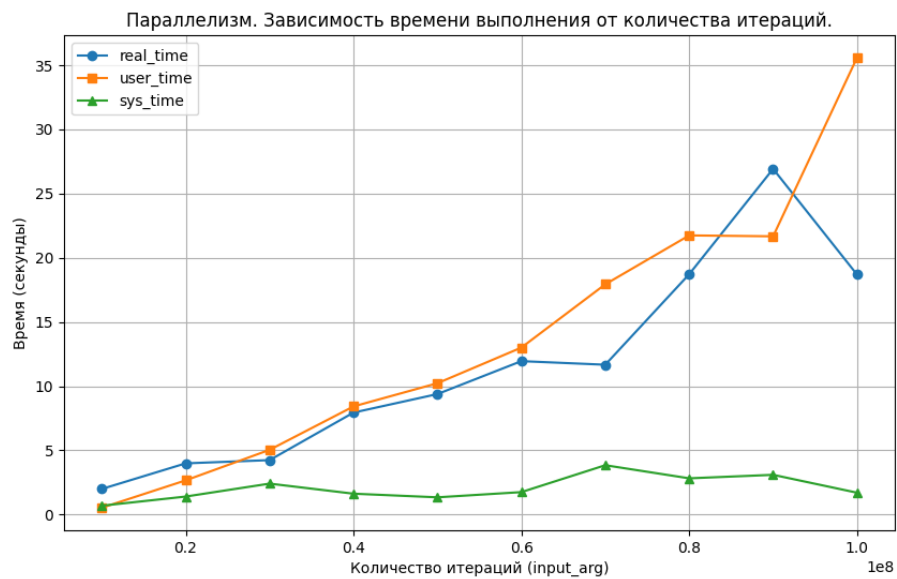
## График при конкурентных вычислениях:



## timetest\_parallel

```
sep=;
input_arg;real_time;user_time;sys_time
10000000;1.995;.531;.691
20000000;3.990;2.665;1.404
30000000;4.242;5.043;2.412
40000000;7.945;8.424;1.627
50000000;9.387;10.232;1.344
60000000;11.952;13.007;1.750
70000000;11.673;17.938;3.845
80000000;18.723;21.746;2.824
90000000;26.929;21.675;3.097
100000000;18.693;35.591;1.704
```

## График при параллельных вычислениях:



## **Замеры.**

Сравнивая два графика можно заметить, что параллельные вычисления зачастую дают выигрыш в скорости, но случается такое что выигрыша не происходит.

Также при параллельных вычислениях заметно, что имеется расход времени на системные нужды. Тогда как при конкурентных вычислениях затраты практически отсутствуют.

При конкурентных вычислениях чётко прослеживается линейная зависимость кол-ва итераций от времени. Тогда как при параллельных вычислениях возникают «артефакты» в виде скачков времени выполнения.

## **Выводы**

В ходе лабораторной работы было выполнено индивидуальное задание. Исследована чувствительность вычислительной схемы из индивидуального задания к ситуациям конкурентности и параллелизма.

Параллелизм оказываются эффективными для больших вычислений на многоядерном процессоре за счёт разделения ресурсов. Конкурентная среда не расходует время работы ядра системы поэтому для небольших вычислений будет эффективно использовать такой подход.